Case Study 29: Password Policy Validation Engine

1. Problem Statement

Evaluate sample password strings against minimum length, character-category rules (uppercase, lowercase, digits, special characters), consecutive repetition limits, and prohibited substring conditions; produce a detailed audit explaining each failed rule.

2. Objectives

- Validate candidate passwords using basic string traversal, checks, and conditional logic.
- Enforce individual security constraints without relying on external regex libraries.
- Provide diagnostic feedback for every violated constraint and assign a security status rating.

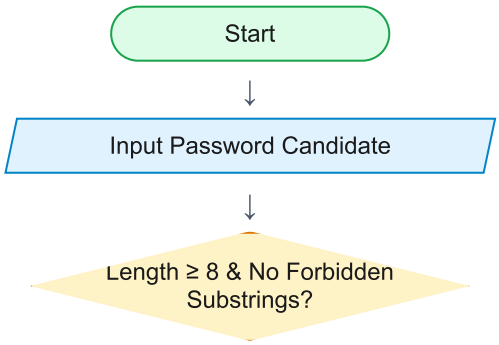
3. Assumptions

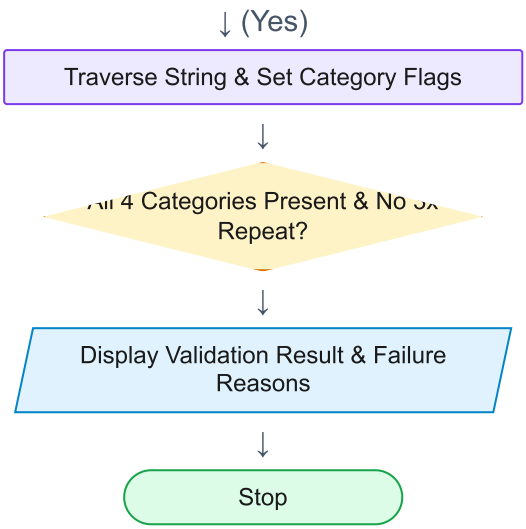
- Minimum password length is **8 characters**.
- Must contain at least one uppercase letter (`A-Z`), one lowercase letter (`a-z`), one digit (`0-9`), and one special character from `@#$%^&+=!`.
- Maximum allowed consecutive identical characters is **2** (e.g., "AAA" is invalid).
- Prohibited substrings include `"1234"`, `"password"`, and `"admin"` (case-insensitive).

4. Algorithm

1. Start.
2. Read the candidate password string.
3. Initialize flags: `has_upper = False`, `has_lower = False`, `has_digit = False`, `has_special = False`.
4. Check length: If `len(password) < 8`, record rule failure.
5. Iterate through each character to set character-category flags and detect 3 consecutive identical characters.
6. Check for prohibited substrings (`password.lower()` containing prohibited phrases).
7. If all rule checks pass, report **VALID / STRONG**; otherwise, list all failed conditions.
8. Stop.

5. Flowchart





6. Pseudocode

```
START INPUT password errors = [] IF length(password) < 8 THEN APPEND "Length less than 8 characters" TO errors ENDIF IF contains_prohibited(password) THEN APPEND "Contains prohibited substring" TO errors ENDIF has_upper = False has_lower = False has_digit = False has_special = False FOR i FROM 0 TO length(password) - 1 DO c = password[i] IF is_uppercase(c) THEN has_upper = True ENDIF IF is_lowercase(c) THEN has_lower = True ENDIF IF is_digit(c) THEN has_digit = True ENDIF IF is_special(c) THEN has_special = True ENDIF IF i >= 2 AND password[i] == password[i-1] AND password[i-1] == password[i-2] THEN APPEND "More than 2 consecutive identical characters" TO errors ENDIF ENDFOR IF NOT has_upper THEN APPEND "Missing uppercase letter" TO errors ENDIF IF NOT has_lower THEN APPEND "Missing lowercase letter" TO errors ENDIF IF NOT has_digit THEN APPEND "Missing numeric digit" TO errors ENDIF IF NOT has_special THEN APPEND "Missing special character" TO errors ENDIF IF errors IS EMPTY THEN DISPLAY "PASSWORD VALID / STRONG" ELSE DISPLAY errors ENDIF STOP
```

7. Complete Python Source Code

```
      # Password Policy Validation Engine def validate_password(password):
special_chars = "@#$%^&+=!" prohibited_list = ["1234", "password", "admin"] errors
= [] # Rule 1: Minimum Length if len(password) < 8: errors.append("Must be at least
8 characters long.") # Rule 2: Prohibited Substrings pwd_lower = password.lower()
for item in prohibited_list: if item in pwd_lower: errors.append(f"Contains
prohibited phrase '{item}'.") # Rule 3: Character Categories & Repetitions
has_upper = False has_lower = False has_digit = False has_special = False
has_triple_repeat = False for i in range(len(password)): char = password[i] if
char.isupper(): has_upper = True elif char.islower(): has_lower = True elif
char.isdigit(): has_digit = True elif char in special_chars: has_special = True if
i >= 2 and password[i] == password[i-1] == password[i-2]: has_triple_repeat = True
if not has_upper: errors.append("Missing at least one uppercase letter (A-Z).") if
not has_lower: errors.append("Missing at least one lowercase letter (a-z).") if not
has_digit: errors.append("Missing at least one numeric digit (0-9).") if not
has_special: errors.append(f"Missing at least one special character
({special_chars}).") if has_triple_repeat: errors.append("Exceeds 2 consecutive
identical characters.") return errors # Test Execution test_passwords = [
"Pass123!", "admin2024!", "weak", "AAAbc123#", "Secure#Pass2026" ] print("=" * 65)
print(" PASSWORD POLICY VALIDATION ENGINE REPORT") print("=" * 65) for pwd in
test_passwords: errs = validate_password(pwd) print(f"Candidate Password : \"
{pwd}\"") if not errs: print("Status : VALID (Meets all security criteria)") else:
print("Status : INVALID") for err in errs: print(f" └─ Failed Rule : {err}")
print("-" * 65)
```

8. Test Case and Expected Output

Candidate Password Audit Output:

```
      ================================================================= PASSWORD
POLICY VALIDATION ENGINE REPORT
================================================================= Candidate
Password : "Pass123!" Status : INVALID └─ Failed Rule : Contains prohibited phrase
'1234'. ----------------------------------------------------------------- Candidate
Password : "admin2024!" Status : INVALID └─ Failed Rule : Contains prohibited
phrase 'admin'. └─ Failed Rule : Missing at least one uppercase letter (A-Z). -----
------------------------------------------------------------ Candidate Password :
"weak" Status : INVALID └─ Failed Rule : Must be at least 8 characters long. └─
Failed Rule : Missing at least one uppercase letter (A-Z). └─ Failed Rule : Missing
at least one numeric digit (0-9). └─ Failed Rule : Missing at least one special
character (@#$%^&+=!). ------------------------------------------------------------
----- Candidate Password : "AAAbc123#" Status : INVALID └─ Failed Rule : Exceeds 2
consecutive identical characters. ----------------------------------------------
---------------- Candidate Password : "Secure#Pass2026" Status : VALID (Meets all
security criteria) ------------------------------------------------------------
-
```

9. Conclusion

The Password Policy Validation Engine demonstrates precise string auditing by evaluating length, character sets, repetitive sequences, and forbidden substrings, outputting clear diagnostic feedback to guide users in creating strong credentials.